

Select then Match ? a new LLM-powered Entity Matching Strategy

AIDM Seminar 2025/2026 Report

Daril Vergesse Sobze Soffack

1 TASK & PAPER

Entity Matching (EM) is a critical data engineering step that aims to identify records referring to the same real-world entity from a set of potential matches filtered by a blocking step. The system must either determine the correct match or decide that no match exists among the candidates. We focus on the paper "Match, Compare, or Select? An Investigation of Large Language Models for Entity Matching" [1]. The authors investigate three distinct prompt strategies (matching, comparing, and selecting) for LLM-based EM to incorporate global consistency and record interactions (Figure1). Furthermore, they propose the COMEM framework, which sequentially applies local ranking and global selection to balance both effectiveness and computation cost (Figure 2).

Do these two records refer to the same real-world entity?

(1) Title: *Cruzer Force USB Flash Drive 32GB Type-A 2.0 Chrome*

(2) Title: *Sandisk USB Flash Drive 32GB Cruzer Glide 2.0/3.0*

LLM Response No

(a) Matching

Which of these two records is more consistent with the given record:

Title: *Cruzer Force USB Flash Drive 32GB Type-A 2.0 Chrome*

(A) Title: *Sandisk USB Flash Drive 32GB Cruzer Glide 2.0/3.0*

(B) Title: *Pendrive Sandisk Cruzer Force - SDCZ71-032G-B35*

LLM Response Record B

(b) Comparing

Select a record from the following list that refers to the same real-world entity as the given record:

Title: *Cruzer Force USB Flash Drive 32GB Type-A 2.0 Chrome*

(1) Title: *Sandisk USB Flash Drive 32GB Cruzer Glide 2.0/3.0*

(2) Title: *Pendrive Sandisk Cruzer Force - SDCZ71-032G-B35*

(3) Title: *Sandisk Extreme Pro 3.1 Solid State Flash Drive 128GB*

(4) Title: *Kingston DataTraveler G4 32 GB USB-stick*

...

LLM Response Record 2

(c) Selecting

Figure 1: Matching, Comparing, Selecting: The Three Strategies for LLM-based for Entity Matching proposed by the paper

2 REPRODUCIBILITY STUDY

We reproduced Table 4 from [1], which compares the F1 scores of LLMs for the eighth standard EM benchmark datasets (Abt-Buy(AB), Amazon-Google(AG), DBLP-ACM(DA), DBLP-Scholar(DS), IMDB-TMDB(IM), IMDB-TVDB(IV), TMDB-TVDB(TT), and Walmart-Amazon(WA)) on four strategies: Matching, Matching 6-shot, Comparing and Selecting.

My evaluation setup follows the original methodology, utilizing the same pre-defined blocks as the paper [1], but We substituted the original models with GPT-5-Mini and only used 100 instances per datasets .

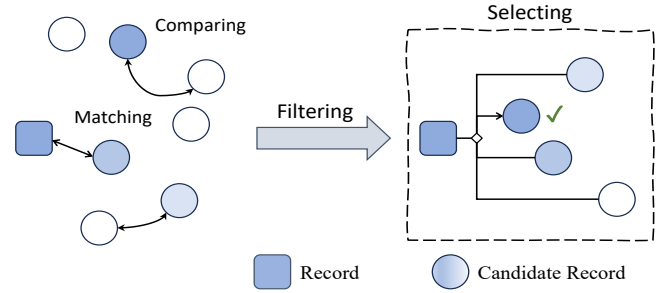


Figure 2: Illustration of ComEM It first filters candidate records by matching or comparing strategies and then identifies the match via the selecting strategy.

Common General Findings: My reproduced results confirm the general findings of the original authors:

- Strategies incorporating multiple records (Comparing and Selecting) generally outperform independent Matching.(Figure3)
- Few-shot Matching improves upon zero-shot Matching.(Figure4)
- Selecting is the most cost-efficient strategy as it requires only $O(1)$ invocations. (Table1)

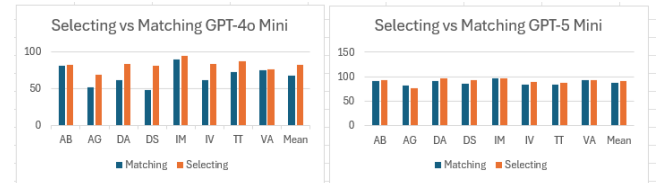


Figure 3: Multiple records-based Strategies (Selecting) vs Matching(pairwise)

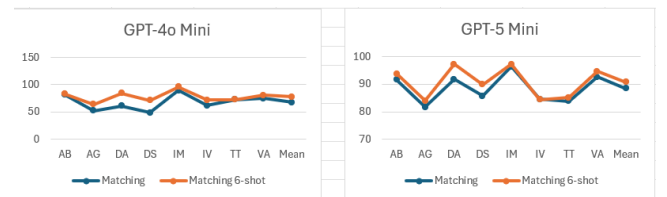


Figure 4: 6-shot Matching performs better than zero-shot Matching

Model Comparison: GPT-4o-Mini vs. GPT-5-Mini: GPT-5-Mini achieved higher F1-scores across all baseline strategies, easily justifying its slightly higher API costs.(Figure5)

	AB	AG	DA	DS	IM	IV	TT	WA	Mean	Total Cost
<i>GPT-5 Mini</i>										
Matching	91.55	81.63	91.7	85.74	96.4	84.58	83.79	92.64	88.50	14.71
Matching (6-shot)	93.72	83.98	97.17	89.85	97.23	84.41	85.14	94.64	90.77	15.65
Comparing	92.69	83.36	97.56	93.96	95.37	90.34	88.73	91.49	92.77	26.2
Selecting	92.19	86.36	96.56	93.36	96.37	89.68	86.73	93.98	90.73	1.49

Table 1: Overall reproduced performance and cost of different methods.

However, the cost distribution diverged significantly between the two models. In the original paper, Matching (6-shot) was by far the most expensive strategy. In contrast, our GPT-5-Mini experiments revealed that Comparing incurred the highest costs.(Table1)

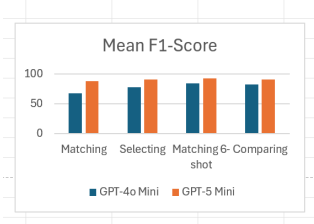


Figure 5: Mean F1-Score of both LLMs GPT-4o-Mini and GPT-5-Mini

Table 2: Total and mean cost of each strategy with GPT-4o-Mini and GPT-5-Mini

Cost	Matching		Matching-6-shot		Comparing		Selecting	
	Mean	Total	Mean	Total	Mean	Total	Mean	Total
GPT-4o-Mini	0.46	3.68	3.21	25.68	1.21	9.68	0.17	1.36
GPT-5-Mini	1.83	14.71	1.96	15.65	3.275	26.2	0.19	1.49

3 APPROACH

3.1 Limitations of the COMEM Framework

The original paper proposes the *COMEM* framework, which integrates the advantages of different strategies through a filtering-then-identifying pipeline. It first utilizes a local matching or comparing strategy to rank potential candidates, and then applies a global selecting strategy to identify the final match.

However, when applying this to modern, highly capable LLMs like GPT-5-Mini, we identified two major architectural challenges:

- **The Redundancy of Pre-Filtering:** COMEM uses initial filtering with the *Matching* and *Comparing* strategies to reduce the job intensity of the selecting strategy in long contexts. However, performing pairwise comparing requires $O(kn)$ LLM invocations. Furthermore We found that GPT-5-Mini is capable of processing the entire block of candidates

directly via the Selecting strategy. Skipping the pre-filtering step and moving directly to a list-wise prompt is significantly more cost-effective, requiring only a single $O(1)$ invocation per anchor record.

- **The “False Positive” Bias in Selection:** The standard Selecting strategy requires the LLM to perform two cognitive jobs simultaneously: look for the candidate, that refers to the same real world entity; if it exists, select his index, otherwise select index 0. In our experiments, we observed that LLMs struggle with the latter. Due to their alignment to be “helpful,” they exhibit a strong bias toward returning an answer rather than definitively rejecting the entire list. This leads to forced selections and a high rate of False Positives based on minor, irrelevant similarities.

3.2 The “SELECT-THEN-MATCH” Pipeline

To address these gaps, We propose an inverted and extended compound pipeline: “**Select-The-Most-Similar-Then-Verify**.” The core idea behind my approach is to decouple the ranking of candidates from the strict verification of a match. We execute this in two sequential steps:

- (1) **Selecting the Most Similar (Filtering):** We modify the Selecting prompt to simply ask for the single most similar candidate from the entire block, removing the option to choose [0]. This relieves the LLM of the verification burden and leverages its ability to consider global context in a single, cheap $O(1)$ invocation (Figure 6).
- (2) **6-Shot Matching (Verification):** We then extract this single chosen candidate and apply a rigorous 6-shot Matching strategy exclusively to that candidate and the anchor record. We deliberately chose the 6-shot variant over zero-shot matching because our reproducibility experiments demonstrated it yields significantly better F1 scores (Figure 4). Because we only run this heavy prompt once per block (on the top-1 candidate) rather than on every pair, the superior accuracy comes at a negligible price—incurring a surplus of only \$1.00 across the evaluation (Table 1).

By using Selecting purely as a top-1 retriever and delegating the final strict “Yes/No” decision to a highly accurate 6-shot pairwise matcher, We successfully eliminate the False Positives caused by the LLM’s reluctance to output “no match.”.

Select then Match ? a new LLM-powered Entity Matching Strategy

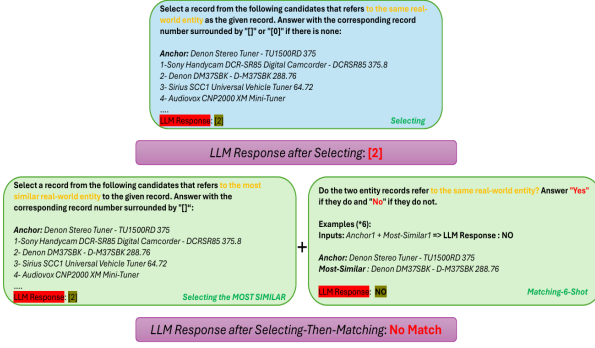


Figure 6: Illustration of my SELECT-THEN-MATCH Strategy. Select first the MOST-SIMILAR real world Entity from the candidates , then Verify if it's a perfect match

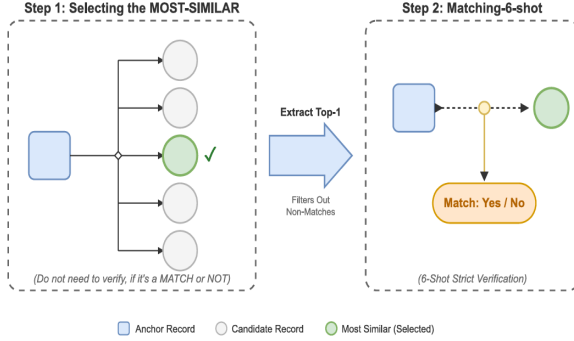


Figure 7: SELECT-THEN-MATCH Architecture

4 EXPERIMENTS

We evaluate my "SELECT-THEN-MATCH" approach using the setup and eight datasets from our reproducibility study. My primary goal was to eliminate the False Positives generated when the standalone Selecting strategy of COMEM is forced to output if there is a match and select it. Table 4 compares our "SELECT-THEN-MATCH" pipeline against the baselines and the COMEM framework.

Comparison with Baseline Strategies. My method leverages the general finding that strategies incorporating multiple records simultaneously perform better by capturing global context and used the Selecting Strategy as first step. This results in significantly higher F1-scores than the others (Table 3)

Comparison with COMEM. By using list-wise selection to just rank candidates and doesn't confirm directly if the chosen candidate is the match , and then applying a strict 6-shot verification to the top choice, our pipeline successfully filters out incorrect forced selections. This results in significantly higher F1-scores than the others (Table 4)

Method	Mean F1-Score	Total Cost (USD)
Matching	88.50	14.71
Matching-6-shot	90.77	15.65
Comparing	92.77	26.20
Selecting	90.73	1.49
Select-Then-Match	95.49	3.16

Table 3: Overall Performance and Cost for baseline strategies and Select-Then-Match.

Method	Mean F1-Score	Mean Cost (USD)
COMEM [1]	86.56	0,09
SELECT-THEN-MATCH	95,49	0,395

Table 4: Overall Performance and Cost. Compare Mean F1-Score and Total Cost (USD) for COMEM and SELECT-THEN-MATCH. Note that the costs of COMEM are so low , because it used a medium-sized LLM for the filtering (Matching/comparing

5 CONCLUSION

We have shown that my "Select-Then-Verify" pipeline effectively bridges the gap between global context awareness and strict matching verification. By using the Selecting strategy purely for ranking and relying on 6-shot Matching for the final decision, my approach successfully improves precision by preventing the LLM from forcing incorrect matches

However, the 6-shot verification phase remains computationally expensive due to high token consumption and reliance on large commercial models. Because this final step evaluates only a single pair of records, it is a straightforward binary classification task that does not require a massive foundation model.

Future work should explore replacing the large model with a fine-tuned Small Language Model (SLM). Fine-tuning models for better execution of specific strategies is a worthwhile direction. A task-specific SLM would maintain high accuracy while making the pipeline significantly faster and more cost-efficient.

REFERENCES

- [1] Tianshu Wang, Xiaoyang Chen, Hongyu Lin, Xuanang Chen, Xianpei Han, Hao Wang, Zhenyu Zeng, and Le Sun. 2024. Match, Compare, or Select? An Investigation of Large Language Models for Entity Matching. arXiv:2405.16884 [cs.CL] <https://arxiv.org/abs/2405.16884>